System Programs and OS Structures

Lecture 4 — Monolithic, Layered, Microkernel, Modular, Virtual Machine

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Content Not Click

Welcome: Where Does a Driver Bug Land?

The big question

A network-card driver has a bug: on a rare packet it dereferences a `NULL` pointer. Does the whole machine crash, does only the network stop, or does nobody notice?

- The answer is *not* in the driver's code. It is in how the operating system is **organized**.
- Where the driver runs — kernel mode or user mode, inside the kernel or beside it — decides how far the damage spreads.
- Last lecture: the system-call doorway. Today: what sits on each side of it.

Running thread for today

We drop the same buggy driver into five structures and watch the blast radius change.

Roadmap: Above the Doorway, Below the Doorway

1. **System programs** — the user-level tools built on system calls.
2. **Design principles** — goals, and separating mechanism from policy.
3. **Kernel structures** — simple, monolithic, layered, microkernel, modular.
4. **Virtual machines** — an operating system as a guest of another.
5. **Comparison** — one table, one driver bug, five outcomes.

Silberschatz et al., Ch. 2 §2.5–2.8, pp. 66–84; Stallings, Ch. 2 and Ch. 4 §4.3.

Concepts That Click

Most of what a user calls “the operating system” is not the kernel at all. It is a set of ordinary programs that make system calls.

What Is a System Program?

Definition

A **system program** (system utility) is a program shipped with the operating system that gives users a convenient environment for program development and execution. It runs in *user mode* and obtains every privileged service through system calls.

- `ls`, `cp`, `cc`, `vi`, the shell, the login program — all are system programs.
- Some are thin wrappers around one system call; others are large programs in their own right.
- The user's view of the OS is defined by the system programs, not by the system calls.

Silberschatz et al., Ch. 2 §2.5, p. 66.

Concepts TM

The Categories of System Programs

Category	What it does	Examples
File management	create, delete, copy, rename, list files and directories	`cp`, `mv`, `rm`, `ls`
Status information	report date, time, memory, users, performance	`date`, `who`, `df`
File modification	create and change file contents	`vi`, `grep`, `sed`
Programming-language support	compilers, assemblers, interpreters, debuggers	`cc`, `as`, `gdb`
Program loading and execution	loaders, linkage editors	`ld`, the loader
Communications	connections between processes, users, and machines	`ssh`, `mail`, `ftp`

Concepts That Click

- Beyond these, **application programs** (browsers, word processors, databases) ship with many systems but are not part of the OS proper.

Categories per Silberschatz et al., Ch. 2 §2.5, p. 66.

Worked Example: Classify Six Commands

1. `ls -l` — **file management**: lists a directory's entries (uses `open`, `getdents`-style reads, `stat`).
2. `date` — **status information**: reads the clock through an information-maintenance system call.
3. `vi notes.txt` — **file modification**: edits contents with `read` and `write`.
4. `cc prog.c` — **programming-language support**: compiles source into an object file.
5. `ld prog.o` — **program loading and execution**: links objects into an executable.
6. `ssh server` — **communications**: opens a connection with `socket`.

Note the two taxonomies: a system *program* category (above) is not the same as a system *call* category (Lecture 3). `date` is a status-information program that makes an information-maintenance call.

Concepts That Click

The Command Interpreter: Two Designs

- The **shell** reads a command and gets it executed. Two ways to build it:

Built-in commands

The interpreter itself contains the code for each command. Adding a command means changing the interpreter. The interpreter grows with every command.

Commands as system programs (UNIX)

`rm file.txt` makes the shell find a *file* named `rm`, load it, and pass `file.txt` as an argument. New commands are new files; the shell stays small.

The UNIX shell uses the second design for most commands (and keeps a few built-ins such as `cd`). Silberschatz et al., Ch. 2 §2.2.

Socratic Check 1

Question

`cd` must be a shell built-in, not a separate program. Why? (Hint: which process's current directory would a separate `cd` change?)

- A separate program runs in a *child* process created by `fork`.
- `chdir` in the child changes only the child's current directory; the child then exits.
- The shell's own directory is unchanged — so the shell must call `chdir` itself.

Concept Check

Before choosing a structure, decide what the system is for — and keep the decisions about *what* to do apart from the machinery for *how* to do it.

Design Goals, Mechanism, and Policy

- **User goals**: convenient, easy to learn, reliable, safe, fast.
- **System goals**: easy to design, implement, and maintain; flexible, reliable, error-free, efficient.
- The goals conflict; there is no single "best" OS, only a best fit to a set of requirements.

Definition: mechanism vs. policy

A **mechanism** determines *how* to do something; a **policy** determines *what* will be done. Separating them lets a policy change without rewriting the mechanism.

Silberschatz et al., Ch. 2 §2.6.

Worked Example: Separate the Mechanism From the Policy

Situation	Mechanism (how)	Policy (what)
CPU protection	the timer interrupts the CPU after a set interval	how long the interval is for each user
Scheduling	a context switch from one process to another	which process runs next
File protection	the kernel checks a permission bit before `open`	who is granted which bits
Our driver	loading code into the kernel at run time	which drivers are trusted enough to load

Why the split pays

The timer mechanism (Lecture 2) is unchanged whether the quantum is 10 ms or 100 ms; only a parameter changes. A microkernel pushes this idea to its limit: the kernel keeps mechanisms, servers hold policy.

Implementation: Which Language?

- Early operating systems were written in **assembly language**.
- Modern kernels (Linux, Windows) are written mostly in **C**, with small parts in assembly: the trap entry, context switch, and device-register access.
- A high-level language: faster to write, easier to read and debug, easier to **port** to new hardware.
- The cost: possibly slower code — but the bottlenecks are usually algorithms and data structures, which can be tuned after profiling.

Takeaway

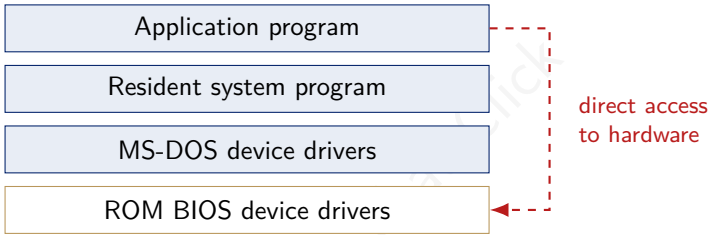
The structure of the code matters more than its language: a well-structured C kernel can be optimized where it counts.

Silberschatz et al., Ch. 2 §2.6.

A kernel is millions of lines. How they are divided into parts — and which parts share kernel mode — is the structure.

Concepts Click

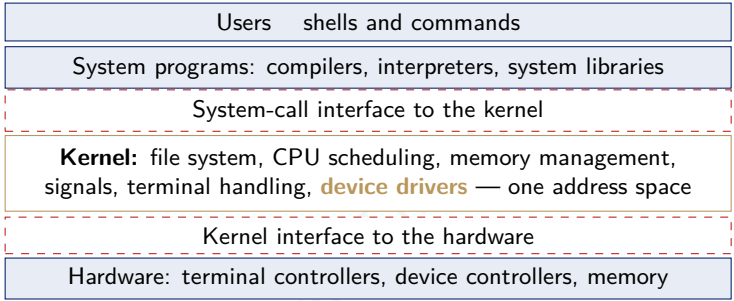
Simple Structure: MS-DOS



- Written for small machines; levels are not well separated.
- Applications can call BIOS routines and write to the display directly — the Intel 8088 had **no dual mode** and no hardware protection.
- One bad program can crash the whole system.

Silberschatz et al., Ch. 2 §2.7, pp. 70–76.

Monolithic Structure: Traditional UNIX



- Two parts: the **kernel** and the **system programs**.
- Everything below the system-call interface is *one* program in kernel mode: fast (a function call between subsystems), hard to maintain.

Silberschatz et al., Ch. 2 §2.7, pp. 70–76.

Running Example 1: The Bad Driver in a Monolithic Kernel

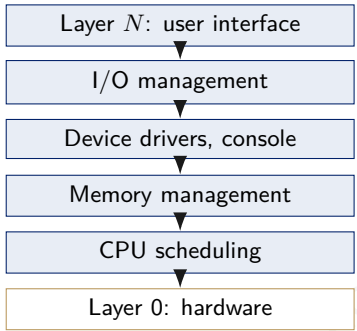
1. The driver is compiled into the kernel and runs in **kernel mode** in the kernel's address space.
2. The rare packet arrives; the driver dereferences `NULL`.
3. The fault happens in kernel mode — there is no more-privileged code to contain it.
4. The kernel cannot tell which of its data structures the driver corrupted, so it halts (a *kernel panic*).

Outcome

Blast radius: the whole machine. Every process, not just the network, is lost.

The same speed that makes monolithic kernels attractive — no boundary between subsystems — is what lets one bug reach everything.

Layered Approach



- The OS is a stack of **layers**; layer i uses only the operations of layers below i.
- Debug bottom-up: once layer $i-1$ is verified, a bug found while testing layer i must be in layer i.
- Each layer hides its data structures from the layers above.

Silberschatz et al., Ch. 2 §2.7, pp. 70–76; Stallings, Ch. 2 §2.3, p. 64.

Concepts That Old

Worked Example: Where Does the Backing-Store Driver Go?

- **Rule:** if A calls B, then B must be in a *lower* layer than A.

1. Memory management swaps pages to disk ⇒ it calls the backing-store driver ⇒ **driver below memory management**.
2. The driver waits for I/O, and the CPU should run someone else meanwhile ⇒ it calls the CPU scheduler ⇒ **scheduler below driver**.
3. But on a large system the scheduler's tables may not fit in memory and must be swapped ⇒ the scheduler calls the driver ⇒ **driver below scheduler**.

Result

Steps 2 and 3 demand opposite orders. Choosing the layers requires careful planning, and some dependencies have no clean order at all.

Layered Approach: The Costs

- **Defining the layers is hard** — the previous slide's cycle is typical.
- **Each layer adds overhead**: a request from the top passes through every layer below it, each adding a call and possibly copying parameters.
- A system call that needs I/O traverses I/O → memory → scheduling → hardware, even when a direct path would do.

Running Example 2: the bad driver, layered

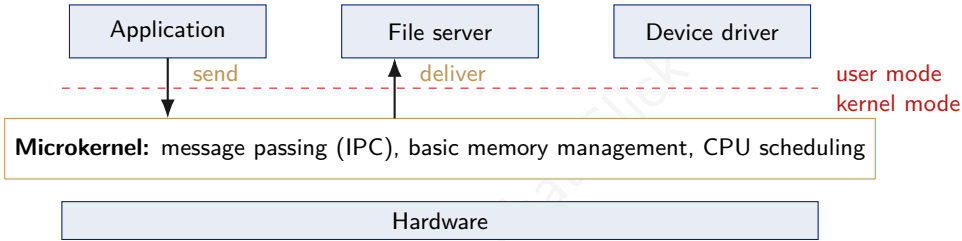
Layering organizes code; it does not change privilege. The driver layer still runs in kernel mode, so the `NULL` dereference still crashes the whole kernel. The win is finding the bug sooner, not surviving it.

Modern systems use fewer, more functional layers to keep the benefits without the cost.

If kernel mode is where one bug becomes everyone's bug, put as little as possible in kernel mode.

Concepts Click

Microkernel Structure



- Move everything nonessential — file systems, drivers, networking — into **user-level servers**.
- Clients and servers never call each other directly: they exchange **messages** through the microkernel.

Mach (CMU, mid-1980s), QNX. Silberschatz et al., Ch. 2 §2.7; Stallings, Ch. 4 §4.3, p. 179.

Worked Example: Count the Boundary Crossings

Task: the application reads a file block. Count traps (each trap = one user→kernel entry + one kernel→user return = 2 crossings).

1. **Monolithic:** one `read` trap; the file system and driver are kernel functions. ⇒ **1 trap, 2 crossings**.
2. **Microkernel, file server only:** app → server (send: 1 trap), server → app (reply: 1 trap). ⇒ **2 traps, 4 crossings**.
3. **Microkernel, file server + driver server:** app→fs, fs→drv, drv→fs, fs→app. ⇒ **4 traps, 8 crossings**.

General pattern (our counting model)

A request that passes through a chain of k servers and back costs $2k$ message traps, i.e. $4k$ crossings, versus 2 crossings for a monolithic call. With $k = 2$ that is four times as many.

Each message may also be copied between address spaces — the "system-function overhead" that makes microkernels slower.

Microkernel: Benefits, Costs, and the Bad Driver

Benefits

- Extensible: a new service is a new user-level server; the kernel is untouched.
- Portable: little hardware-specific code in the kernel.
- Reliable and secure: most services run as user processes.

Cost

- Performance: messages and extra crossings for every service request.

Running Example 3

The driver is a *user process*. The `NULL` dereference causes an ordinary page fault; the kernel terminates that process only, and can restart it.

Blast radius: the network driver. Other processes keep running.

Stallings, Ch. 4 §4.3, p. 179 lists further benefits (uniform interfaces, flexibility, distributed-system support).

Socratic Check 2

Question

Windows NT began with a layered microkernel design and later moved more services back into the kernel. What problem were its designers most likely solving?

- The cost of the microkernel design: every service request became messages and extra mode switches.
- Moving services into kernel space turns messages back into function calls — faster.
- The price is the isolation the previous slide bought: a bug in a moved service can again crash the kernel.

Silberschatz et al., Ch. 2 §2.7 discusses this history.

Loadable Kernel Modules

- A small **core kernel** plus **loadable modules** linked in at boot time or at run time — without recompiling the kernel.
- Each module has a defined interface, like a layer — but any module may call any other, like a microkernel's servers — yet no messages are needed, because all run in kernel mode.

Solaris: core kernel and seven module types

scheduling classes ▪ file systems ▪ loadable system calls ▪ executable formats ▪ STREAMS modules ▪ miscellaneous ▪ device and bus drivers

Silberschatz et al., Ch. 2 §2.7, pp. 70–76. Linux is monolithic with loadable modules: Bovet & Cesati, Ch. 1, pp. 1–34.

Hybrid Systems and the Bad Driver as a Module

- Real kernels mix structures to balance speed against isolation:
  - **Linux**: monolithic, plus loadable modules.
  - **Mac OS X**: a Mach microkernel combined with a BSD kernel environment, plus loadable kernel extensions.
  - **Windows**: largely monolithic for performance, with microkernel-style user-mode subsystems.

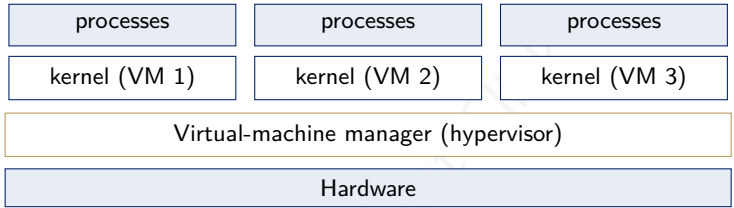
Running Example 4: the bad driver as a module

`insmod netdrv.ko` links the driver into the running kernel — in **kernel mode**. The `NULL` dereference crashes the kernel, exactly as in the monolithic case. Modules buy *flexibility* (load, unload, update without reboot), not *isolation*.

Push the isolation idea one level further: run the entire operating system as a guest, on hardware that is itself simulated.

Concepts Click

The Virtual-Machine Idea



- The **virtual-machine manager** gives each guest the illusion of its own processor, memory, and devices.
- Each guest runs a full, unmodified operating system — possibly a different one in each VM.

IBM VM/370 (1972), VMware, Java Virtual Machine. Silberschatz et al., Ch. 2 §2.8, p. 76.

Concepts Tutorial

Virtual Machines: Benefits and the Bad Driver

- Isolation: each VM is protected from the others; a virus in one cannot reach another directly.
- System development: test a new kernel in a VM without disrupting normal work on the same machine.
- Consolidation: several lightly used servers become VMs on one physical machine.
- Cost: the guest kernel itself runs in a de-privileged mode, so privileged operations must be trapped and emulated by the manager.

Running Example 5

The driver runs in the *guest* kernel of VM 1. The guest panics — all of VM 1 is lost — but the manager and VMs 2 and 3 keep running. Blast radius: one virtual machine.

Silberschatz et al., Ch. 2 §2.8, p. 76.

One Driver Bug, Five Structures

Structure	Driver runs in	Bug's blast radius	Speed	Main strength
Monolithic	kernel mode	whole machine	fastest (calls)	performance
Layered	kernel mode	whole machine	layer overhead	bottom-up debugging
Microkernel	user mode	the driver only	messages, $4k$ crossings	isolation, extensibility
Modular	kernel mode	whole machine	near monolithic	load at run time
Virtual machine	guest kernel	one VM	trap-and-emulate	isolates whole OSes

The pattern

Isolation is bought with boundary crossings. Every structure picks a point on that trade-off.

Socratic Check 3

Question

An aircraft flight-control computer must keep running even if its network-logging driver fails. Which structure would you choose, and what performance price do you accept?

- A **microkernel**: the logging driver runs as a user-level server, so its failure cannot take down the control software.
- The price: every driver request is a message — with a chain of k servers, $4k$ crossings instead of 2.
- This is why microkernels such as QNX are common in embedded and real-time systems.

Concepts That

Common Misconceptions

Wrong

- "The shell is part of the kernel."
- "Layered means the upper layers run in user mode."
- "Loadable modules isolate drivers."
- "Microkernels are faster because the kernel is small."

Right

- The shell is a system program in user mode.
- Layering organizes code; privilege is unchanged.
- Modules run in kernel mode; a bug crashes the kernel.
- Messages add crossings; microkernels trade speed for isolation.

Takeaways

- **System programs** (file management, status, file modification, language support, loading, communications) run in user mode on top of system calls.
- Separate **mechanism** (how) from **policy** (what).
- **Monolithic**: fast, one address space, one bug crashes all. **Layered**: clean debugging, hard layer ordering, overhead.
- **Microkernel**: minimal kernel, services as user-level servers, isolation paid for in message crossings.
- **Modular**: run-time flexibility, still kernel mode. **Virtual machine**: isolates entire operating systems.

Concepts of OS

Next: The Process

- Today: what runs above the system-call interface, and how the kernel below it is organized.
- Next lecture: the unit every one of these structures manages — the **process**, and the kernel record that describes it, the **process control block**.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 3 §3.1 [2]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 3 §3.1–3.3 [3].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 2 §2.5 (system programs, p. 66), §2.6 (design and implementation), §2.7 (OS structure, pp. 70–76), §2.8 (virtual machines, p. 76); §2.5–2.8 span pp. 66–84.
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 2 §2.3 (system structure, p. 64) and Ch. 4 §4.3 (microkernels, p. 179).
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 1 (kernel architecture: monolithic with loadable modules, pp. 1–34) [1].

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Concept Not Click

References I


Daniel P. Bovet and Marco Cesati.
Understanding the Linux Kernel.
O'Reilly Media, 3rd edition, 2006.


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click